CT2003 OBJECT-ORIENTED ANALYSIS & DESIGN

Restaurant Rush — Technical Documentation & V3 Roadmap

Version 2.0 (Stable) Stage 3 Requirements

---

1. Architecture Overview

The project follows a decoupled **3-Tier Mediator Architecture**. The primary objective is to separate presentation logic, state tracking, and domain entity behavior to ensure scalable and maintainable execution.

- **Controller Tier (`GameController`):** Acts as the Central Mediator owning all domain logic, step execution, turn management, and transactional state updates.
- **Staff Tier (`Staff` , `Waiter` , `Chef` , `Host` , `Cashier`):** Represents staff capacity state using simple boolean action indicators rather than embedding game logic directly.
- **Customer Tier (`Customer` , `RegularCustomer` , `VIPCustomer` , `CriticCustomer`):** Tracks satisfaction decay and customer states across a 7-stage lifecycle.
- **Domain Tier (`Table` , `Order` , `MenuItem` , `BillCalculator`):** Handles physical restaurant layout and financial calculation rules.

2. Key Features & v2 Technical Improvements

Turn-Start Arrival Display

Queue updates and customer arrivals for scheduled turns (e.g., Turn 1, Turn 4) execute during `startTurnStep1()` , guaranteeing that new arrivals (such as `CUST-1`) render on screen prior to manager input collection.

Atomic Action Validation

Turn execution uses transactional validation via `validateAllActions(...)` . All inputs for Host, Waiter, and Chef choices are validated against simulated turn states prior to applying any mutations. If an invalid move occurs:

- An `InvalidGameActionException` is raised.
- No customer seating, order placement, or prep advancement takes effect.
- Staff boolean capacities are fully preserved for input re-entry.

Same-Turn Action Support

The validation system allows same-turn operation where the Waiter places an order on a table and the Chef cooks that same order within the same turn cycle.

3. Verification Matrix for Happy Hour & Lifecycle Rules

> **Happy Hour Mechanics Checklist:**
>
> - **Activation:** Triggered once per game for a 2-turn duration (`happyHourTurnsRemaining = 2`).
> - **Satisfaction Boost:** Grants `+15` satisfaction (capped at 100) to all customers currently in the `WAITING` queue or `SEATED` state during Turn Step 1.
> - **Pricing Discount:** Applies a 20% discount (`* 0.80`) calculated dynamically via `BillCalculator` .
> - **Persistence:** Flags `happyHourDiscountApplied = true` on `Order` objects taken during Happy Hour to preserve discount benefits through payment.

4. Developer Roadmap for Version 3 (v3)

Task 1: Explicit Turn Reset Option (QoL)

Goal: Allow the player to clear turn selections manually without triggering an exception.

Implementation: Add a reset command (e.g., typing `r` or `9`) in `RestaurantRushCLI` . Since `GameController` defers execution until choices are submitted, catching a reset input simply restarts the input loop without calling `executeRemainingSteps(...)` .

Task 2: Graphical User Interface (GUI Integration)

Goal: Replace or supplement the CLI with a visual UI built in Swing or JavaFX.

Implementation: The decoupled architecture means no changes to domain logic are required. Connect visual buttons directly to `GameController` action methods, and render states using getters like `getWaitingQueue()`, `getTables()`, and `getCurrentTurn()`.

Task 3: Automated Test Suite Coverage

Goal: Implement JUnit tests for edge case validation.

Implementation: Create unit tests targeting `validateAllActions()`, `BillCalculator` discount stacking (VIP + Happy Hour), and satisfaction decay freezing upon reaching the `SERVED` state.
